

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



SIMATS Online Programming Lab

DAA PROGRAMMING



SAKE MYTHILI
192411250RA

ASSESSMENT-CO3/AT2-CASE STUDY

[← Back](#)

Language: Python C C++ Java

QUESTIONS

Q1

Retail Inventory Sorting (Merge vs Quick Sort)
100 marks



1/1 answered

Q1

Retail Inventory Sorting (Merge vs Quick Sort) 100 marks

A retail system must frequently sort inventory data based on price and demand. Analyze the use of Merge Sort and Quick Sort in this context. Identify issues such as memory usage, speed, and data size. Apply divide and conquer principles to compare both algorithms. Analyze their performance and design a solution, justifying the most suitable approach.

Your Code

```
1 import time
2 import random
3 import sys
4 def merge_sort(arr):
5     if len(arr) <= 1:
6         return arr
7     mid = len(arr) // 2
8     left = merge_sort(arr[:mid])
9     right = merge_sort(arr[mid:])
10    return merge(left, right)
11
12 def merge(left, right):
13     result = []
```

```

14     i = j = 0
15     while i < len(left) and j < len(right):
16         if left[i][1] <= right[j][1]: # so
17             result.append(left[i])
18             i += 1
19         else:
20             result.append(right[j])
21             j += 1
22     result.extend(left[i:])
23     result.extend(right[j:])
24     return result
25 def quick_sort(arr):
26     if len(arr) <= 1:
27         return arr
28     pivot = arr[len(arr) // 2]
29     pivot_val = pivot[1]
30     left = [x for x in arr if x[1] < pivot_val]
31     mid = [x for x in arr if x[1] == pivot_val]
32     right = [x for x in arr if x[1] > pivot_val]
33     return quick_sort(left) + mid + quick_sort(right)
34 def generate_inventory(n):
35     # Each item: (item_id, price_or_demand_val)
36     return [(f"Item{i}", random.randint(1, 10000)) for i in range(1, n+1)]
37 def compare_sorts(sizes):
38     print(f"{'Size':<10}{'MergeSort(s)':<15}")
39     for n in sizes:
40         data = generate_inventory(n)
41
42         data_copy1 = data.copy()
43         start = time.time()
44         merge_sort(data_copy1)
45         merge_time = time.time() - start
46
47         data_copy2 = data.copy()
48         start = time.time()
49         quick_sort(data_copy2)
50         quick_time = time.time() - start
51
52         print(f"{n:<10}{merge_time:<15.6f}{quick_time:<15.6f}")
53 if __name__ == "__main__":
54     sys.setrecursionlimit(1000)
55     sample_inventory = [("Laptop", 55000), ("Monitor", 8000), ("Keyboard", 5000), ("Mouse", 1000)]
56     print("Original Inventory:", sample_inventory)
57     merge_sort(sample_inventory)
58     print("Merge Sort Result :", merge_sort(sample_inventory))

```

```
59     print("Quick Sort Result :", quick_sort(  
60     print()  
61     compare_sorts([100, 1000, 5000, 10000])
```

Input

stdin input

Output

```
Original Inventory: [('Laptop', 55000),  
('Mouse', 500), ('Keyboard', 1200),  
('Monitor', 8000), ('Charger', 700)]  
Merge Sort Result : [('Mouse', 500),  
('Charger', 700), ('Keyboard', 1200),  
('Monitor', 8000), ('Laptop', 55000)]
```

✓ Submitted